

DealFlow360

A self-governing B2B sales operations engine — solution architecture and design rationale

[Odoo Hackathon Finals](#) · [Problem statement: Sales operations platform](#) · [Review round 1 submission](#)

This document explains what we are building, how it is architected, the core algorithms behind each governing rule, and — for every major decision — why we chose this approach over the obvious alternative. The goal is a system that governs itself: pricing discipline, inventory reality, and billing correctness are enforced by logic, not by a rep's judgement or a hardcoded rule.

Contents

1. MVP scope and the two flows we go deep on
2. Why this problem needs more than CRUD
3. System architecture — modular monolith, event-sourced
4. Core algorithms and their formulas
5. Data model summary
6. Technology stack and why each piece was chosen
7. Design decisions — what we rejected and why
8. What we would build next

1. MVP scope — two flows, built deep

The problem statement covers 16 feature areas across backend configuration and rep-facing screens. Rather than spreading effort thin across all of them, we take two flows end to end and make every rule inside them real, computed, and demonstrable live — because that is what review criteria and the demo format both reward.

Flow A — Discount governance and approval routing

Rep builds a quote → blended risk score computed per line → quote auto-routes to Manager, or Manager + Finance → approver acts → audit trail entry is generated automatically from the event log, not written by hand.

Flow B — Fulfillment split + hybrid billing

Approved order → warehouse allocation solver splits stock to minimise shipments → one-time and recurring subscription lines are billed on a single order → a mid-cycle quantity change triggers a correctly prorated charge.

Every other backend/config screen (products, price lists, warehouses, reporting filters) is implemented functionally so the system is coherent, but the polish and algorithmic depth are concentrated in these two flows.

2. Why this problem needs more than CRUD

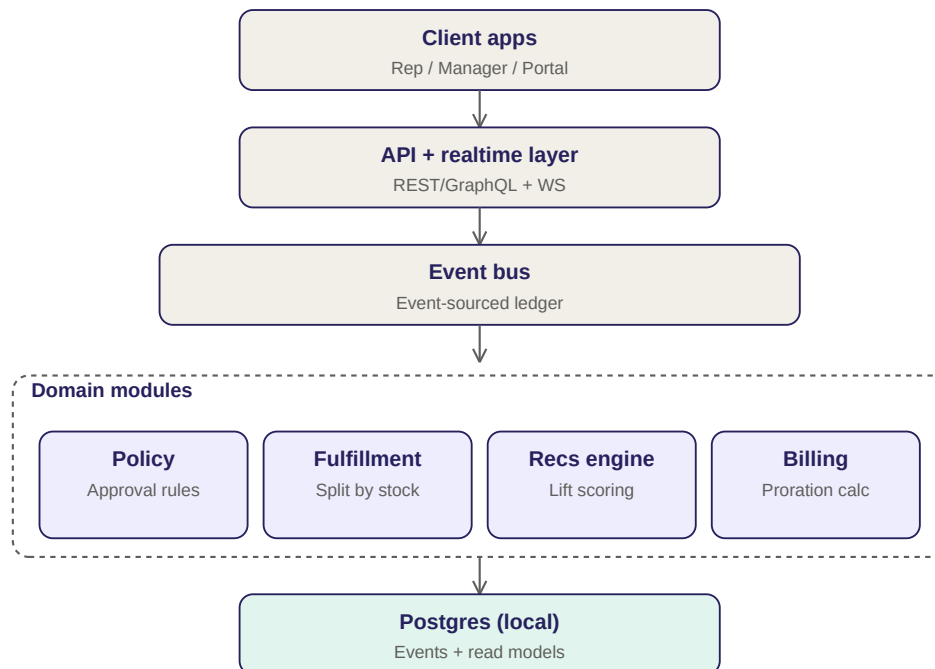
A simple quote-to-invoice form can be built with a handful of database tables and conditionals. DealFlow360 is deliberately harder in three ways that we treat as the actual point of the exercise, not incidental complexity:

- Discount governance is a multi-dimensional constraint problem: different ceilings per category, per tier, aggregated into one order-level decision.
- Fulfillment is a real resource-allocation problem: stock is scattered across warehouses and the split has a cost, not just a correctness requirement.
- Billing must reconcile two fundamentally different revenue shapes (one-time and recurring) on the same order without double-charging or under-refunding.

Because the brief explicitly instructs that these rules must live in application logic and not be hardcoded, the review criteria are effectively asking: can this team build a small rules-and-optimisation engine, not just a form?

3. System architecture

We use a **modular monolith** with domain boundaries drawn like microservices, communicating over an **in-process event bus**, backed by a single local Postgres instance. Every state-changing action is published as a domain event and appended to an event-sourced ledger; each domain module maintains its own read model built by folding over that event stream.



Client apps talk to a single API/realtime layer, which publishes commands as events onto the event bus. Domain modules subscribe to the events relevant to them and update their own tables in Postgres. The ledger itself is also the audit trail.

Why event sourcing instead of a simple audit-log table

A bolt-on audit table records what happened but not why, and it can drift from the actual state if someone edits a row directly. Event sourcing makes the events the source of truth, so the audit trail is a side effect of normal operation rather than a separate thing to remember to maintain. It also buys us a genuine demo differentiator: a **replay/what-if** view that reruns a quote's event history against a different discount-tier configuration and shows what would have happened — something a CRUD app with mutable rows cannot do.

Why a modular monolith instead of real microservices

Microservices would add network calls, service discovery, and deployment complexity that buys nothing at this scale and burns hackathon time on infrastructure instead of logic. A modular monolith keeps the same domain separation and testability, but ships as one deployable unit — the boundaries are enforced by code organisation, not by network boundaries.

4. Core algorithms and their formulas

4.1 Blended discount risk score

Each product line has its own discount ceiling (by category and customer tier). A line's overage is how far its given discount exceeds its own ceiling, as a fraction of that ceiling. The order's blended risk is the value-weighted average overage across all lines, so the score reflects economic exposure rather than a simple headcount of bad lines.

```
line_overage(i) = max(0, (discount_given(i) - ceiling(i)) / ceiling(i))
blended_risk = sum(line_overage(i) * line_value(i)) /
sum(line_value(i))
```

Routing: $\text{blended_risk} = 0 \rightarrow \text{auto-approved}$. $0 < \text{blended_risk} \leq \text{manager_threshold} \rightarrow \text{Sales Manager only}$. Above $\text{finance_threshold} \rightarrow \text{Manager then Finance}$. Thresholds are config values, not code.

Why value-weighted instead of a flat count of over-limit lines: a naive "flag if any line is over" rule cannot distinguish a 1% overage on a Rs.500 line from a 20% overage on a Rs.5,00,000 line. Weighting by line value means the score reflects actual margin risk to the business, which is the thing approval is meant to protect.

4.2 Warehouse fulfillment split

For each order line, stock may be spread across warehouses with different shipping costs. We allocate demand to the cheapest available warehouse first, then the next, until demand is met or stock runs out (the remainder becomes a backorder).

```
allocate(line):
    remaining = qty_needed
    for w in warehouses sorted by shipping_cost(w) ascending:
        take = min(remaining, stock(w))
        allocate take units from w; remaining -= take
        if remaining == 0: break
    if remaining > 0: create backorder(remaining)
```

Why a greedy heuristic instead of a full linear/mixed-integer solver: for a single product line with a fixed per-warehouse cost and no cross-line bundling constraint, greedy-by-cost is provably optimal (it is the classical single-commodity transportation result) — so a solver would add dependency and runtime cost for the same answer. We call this out explicitly as a scoping decision, and flag true multi-line joint optimisation (fewest total shipments across the whole order) as a stretch goal using a proper solver.

4.3 Upsell and cross-sell recommendations

Rather than a static table of "if product A, suggest product B", we compute lift from seeded historical order data and update it incrementally as new orders close, so recommendations are live, not retrained in a batch job.

```
lift(A, B) = ( count(A and B together) * N ) / ( count(A) * count(B) )
```

A lift above 1 means the products are bought together more than chance would predict. Suggestions are ranked by lift, then filtered to keep only pairs above a minimum margin threshold, so a mathematically strong pairing that erodes margin never surfaces.

Why co-occurrence/lift instead of calling an external recommendation API: it is fully local (no third-party dependency or cost), the math is explainable to a judge in one sentence, and it directly satisfies "based on historical co-purchase data" from the brief rather than approximating it with a static rule.

4.4 Deal health: discount anomaly and stalled-deal detection

A discount is anomalous if it is far from that rep's own historical average, measured in standard deviations (a z-score) rather than an arbitrary fixed percentage, since "normal" discounting varies a lot by rep and product mix.

```
z = (discount_given - mean(rep_discount_history)) /  
stdev(rep_discount_history)  
flag if z > 2 (roughly the top 2.5% of that rep's own history)
```

A deal is stalled if it has been inactive longer than a threshold that adapts per pipeline stage (mean stage duration plus 1.5x the interquartile range) rather than one global cutoff, since a Draft quote and a Negotiation-stage quote have very different normal dwell times.

Why per-rep/per-stage statistics instead of one hardcoded threshold: a single fixed number ("flag if >20% discount" or "stalled after 7 days") is exactly the kind of hardcoding the brief penalises, and it produces false alarms for reps or stages whose normal range is genuinely different.

4.5 Subscription proration

A mid-cycle quantity or plan change is charged (or credited) only for the remaining time in the current billing cycle, computed to the day.

```
prorated_charge = unit_price * delta_qty * (days_remaining_in_cycle /  
total_days_in_cycle)
```

Refunds and mid-cycle credits are posted as negative lines in the same event-sourced ledger used for one-time invoice lines, so a single order's financial history — one-time charges, recurring charges, credits — reads as one timeline instead of two systems that need reconciling.

5. Data model summary

Each domain owns its own tables; the event log is the one thing every domain reads and writes through.

Domain	Key tables	Owns
Catalog & pricing	products, price_lists, tiers	product/price truth
Policy	discount_ceilings, approval_chains, events	risk score, routing
Fulfillment	warehouses, stock_levels, allocations	warehouse split
Subscription/billing	plans, billing_schedules, ledger_lines	proration, invoices
Recommendations	co_occurrence_counts, promotions	upsell ranking
Deal health	rep_discount_stats, stage_durations	anomaly + stall flags
Negotiation/portal	portal_sessions, change_requests	customer-facing state

The event log itself: events(id, aggregate_id, type, payload, actor, timestamp) — append-only, and every domain's read-model tables above are rebuilt by folding over this stream.

6. Technology stack and why each piece was chosen

Layer	Choice	Why
Backend	TypeScript / Node (or Python + FastAPI)	Fast to iterate on with AI pairing; strong typing catches domain-logic bugs early
Database	Postgres, self-hosted (Docker)	Relational integrity for financial/inventory data; zero cloud dependency
ORM	Prisma / Drizzle	Type-safe queries; migrations are code, not manual SQL drift
Realtime	WebSocket (ws) or SSE, self-hosted	Live margin/stock/risk updates without a third-party pub-sub service
Frontend	React + TypeScript	Role-based views (rep, manager, finance, admin, customer) from one component library
Event bus	In-process pub-sub (custom, no broker)	Avoids Kafka/RabbitMQ overhead at this scale while keeping the pattern real
Auth	JWT sessions; portal magic-link simulated locally	No email/Auth0 dependency; still demonstrates the real flow
Exports	Local PDF/XLS generation libraries	No cloud export service required

7. Design decisions — what we rejected and why

Rejected: A no-code/low-code rules table with checkboxes

Looks configurable but is really a disguised if/else tree with a UI on top; doesn't hold up when a judge asks to trace exactly how a decision was made. Our policy evaluator is a small interpreter over declarative rule records, so the same code path handles any future rule without new conditionals.

Rejected: A managed cloud database (Supabase/Firebase/Mongo Atlas)

Adds an external dependency, a network hop, and a vendor the judges cannot inspect. A local Postgres instance is fully within our control and satisfies the "local DB" expectation directly.

Rejected: Calling a third-party recommendation or ML API for upsell

Would work, but is a black box we cannot explain or defend in a review, and it's the opposite of "own engineering." Our lift-based engine is transparent and ours.

Rejected: A real message broker (Kafka/RabbitMQ) for the event bus

Correct pattern, wrong scale — the operational overhead (topics, partitions, consumer groups) is not worth it for a single-process app in a time-boxed build. An in-process event emitter with the same event schema keeps the architecture honest without the infrastructure tax.

Rejected: A single global discount ceiling per customer tier

The simplest possible reading of the brief, but it fails the brief's own worked example (a Gold customer with a stricter Service-category ceiling). Per-category ceilings aggregated into a blended score are what the spec is actually asking for.

8. What we would build next

- A true multi-line joint optimisation for warehouse splitting (minimise total shipments across the whole order, not line by line) using a proper MILP solver.
- A small logistic-regression layer over historical approval outcomes to complement the rule-based risk score with a learned risk signal, kept as an advisory score alongside the explainable one, never replacing it.
- Multi-currency and multi-company support, as the brief marks these as bonus scope.
- A/B testing of upsell ranking (lift vs. margin-weighted lift) once real usage data exists.